

OSS Shift

Evidence-Grounded OSS Dependency Decision Intelligence

공개 npm dependency history 에서 실제 선택 · 유지 · 이탈 · 이동 행동을 근거와 한계까지 함께 본다.

현재 프로젝트 판정 GO

Data Feasibility 는 실제 BigQuery profiling 으로 통과. Movement 는 Week 3 specificity validation 까지 Conditional.

문서 구성

Executive Summary	현재 판단, 핵심 가치, 실제 데이터 feasibility
문제와 제품	사용자, 경쟁 포지셔닝, 제품 경계와 Review Action
데이터와 분석 계약	관찰 모 집 단, mainline, Adoption/Retention/Exit, Transition
State & Evidence	수학적 State rule, entropy/concentration, threshold calibration, AI evidence State 를 Tech Lead 의 판단과 evidence flow 로 연결
Product Experience	Spark DAG, partition/join/shuffle/skew, 1/2/4 Worker benchmark
Distributed System	Quality Gate, Go/Pivot, 6 주 WBS, Acceptance Criteria
Validation & Execution	

목차

대목차와핵심목차중심으로구성했다. 본문에서는하위정의와수식을단계적으로설명한다.

목차

Executive Summary	1
문제와제품	2
Problem	2
Competitive Positioning	3
Product Definition	3
데이터와분석계약	4
Observation Contract	4
Actual Data Feasibility	4
Analysis Unit and Mainline	6
Behavior Metrics	6
Transition Contract	8
State & Evidence	9
State Model	9
Time, Baseline and Support	9
Movement Mathematics	10
State Rules	11
Threshold Calibration	13
Evidence Confidence and Ladder	13
AI Evidence Interpreter	14
Product Experience	15
End-to-End Flow	15
Behavior and Why This State	15
Movement and Cohort Compare	16
Review Action Contract	16
Limitations at the Point of Decision	17
Distributed System	18
Distributed Design Principle	18
System Architecture	18
Data Lake and Partition Contract	19
Spark Job DAG	19
Optimization Strategy	20
Scale-Out Benchmark and Correctness	20

Observability	21
Validation & Execution	22
Validation Framework	22
Quality Gates	22
Go / Conditional / Pivot	23
Six-Week WBS	23
Team and Parallel Development	24
Risk and Scope Cut	25
Final Acceptance Criteria	25
Final Demo Narrative	26
Appendix & Sources	27
Metric Dictionary	27
MVP In / Out	28
Source and Verification Notes	28

Executive Summary

장기간운영할신규서비스나핵심기능에 OSS dependency 를도입할때 Tech Lead 는일반적으로 Stars, downloads, release freshness, vulnerability, maintainer activity 와같은신호를본다. 이신호들은중요하지만 공개 **package ecosystem** 에서실제 **dependency** 선택행동이어떻게변하고있는지를직접설명하지않는다.

OSS Shift 가답하려는핵심질문

공개 npm package 들이 A 를계속새로선택하는가? 이미 A 를사용하던 package 는다음 release 에서도 유지하는가? A 가제거될때특정대안이반복적으로함께추가되는가? 아니면여러 destination 으로분산되는가?

OSS Shift 는공개 npm package 의 **version 별 regular dependency declaration history**를분석해 Adoption, Retention, Exit, Transition Candidate 를계산한다. 그뒤 Behavior 와 Movement 를 deterministic rule 로판정하고, 중요한 transition 에한해서실제 GitHub PR/commit evidence 를결합한다.

제품은특정 library 를자동추천하거나미래에죽을 package 를예측하지않는다. 최종적으로지원하는판단은세가 지뿐이다.

Review Action	의미
Proceed to Review	강한 ecosystem shift signal 이없어일반적인기술검토를계속
Compare Alternatives	본격검토전에 functional alternatives 를함께비교할근거가있음
Investigate	근거가부족하거나변화맥락이복잡해추가확인이필요

실제 Data Feasibility 결과

2026-08-20 실제 BigQuery profiling 에서분석조건에해당하는 npm release version 은약 **4,660 만건**, distinct package 는약 **402 만개**였다. 약 **257.9 만 package(64.1%)** 가두개이상의 release 를가지고있어 version-to-version change 분석의기본조건을충족했다. 이번 profiling 조건에서는 PackageVersionsLatest 와 NPMRequirementsLatest 가 (Name, Version) 기준으로모두연결되었다.

Node.js HTTP Client MVP cohort 의여섯 package 도모두관찰가능했다. 가장큰 axios 는약 16.6 만 distinct dependent package, 가장작은 undici 도약 7.5 천 distinct dependent package 에서 regular dependency declaration 이관찰되었다.

현재판정: GO

Data structure 와 cohort support 때문에프로젝트를중단할이유는발견되지않았다. 다만 Movement 는 A removed + B added co-change noise 를통제해야하므로 Week 3 specificity validation 까지 Conditional 로유지한다.

문제와제품

Health score 를하나더만드는것이아니라, 신규 dependency review 전에실제 ecosystem behavior 를보는 Evidence Layer 를제안한다.

Problem

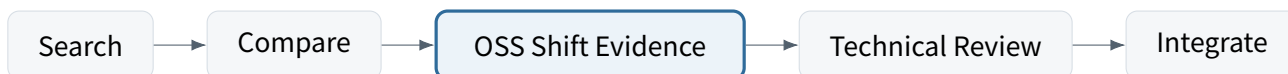
Primary User 와 Decision Context

Primary Persona 는 **Tech Lead / 핵심 dependency 선택을책임지는 Senior Engineer**다. Secondary Persona 는 Software Architect 와 Platform Engineer 다.

Primary 상황은다음과같다.

장기간운영할신규서비스또는핵심기능에, 교체비용이큰 OSS dependency 를도입하기직전

OSS Shift 는기존 dependency 를자동으로교체하는 migration tool 이아니라 **Search/Compare 와본격 Technical Review 사이의 Evidence Layer** 다.



기존지표가직접답하지못하는질문

기존지표는 popularity, security, release freshness, maintainer/community activity 를보는데강하다. OSS Shift 가추가하려는질문은다음과같다.

- A 가 observable published npm packages 에서계속새로추가되는가?
- 기존 adopter 가다음 observable release 에서도 A 를유지하는가?
- A declaration 이사라지는 Exit 가 baseline 대비증가하는가?
- A 가제거될때특정 B 가반복적으로같은 release 에서추가되는가?
- 한 destination 으로집중되는가, 여러 alternatives 로분산되는가?
- 신규유입은크지않아도기존관계가안정적인 Mature Stable 인가?
- same-cohort replacement 가관찰되지않는 No-Replacement Exit 가증가하는가?

핵심은 **Level** 이아니라 **Flow**다.

Competitive Positioning

기존도구가약해서가아니라, **문제축이다르다**. GitHub Dependency Review 는 repository pull request 단 위에서 dependency changes 와 security impact 를검토하는데강하다. SCA/Package Health 계열도구는 vulnerability, maintenance, popularity, supply-chain quality 등 package 상태를폭넓게평가한다. 기존 연구또한 npm dependency change 에서 migration-like pattern 과 alternative 를찾는접근을제시했다.

따라서 OSS Shift 가단순히 A removed + B added 를찾아 B 를추천하면차별성이약하다. OSS Shift 는그보다앞선판단을다룬다.

영역	기존강점	OSS Shift 의질문
Repository Review	PR/commit 단 위 dependency diff, security impact	ecosystem-scale 반복 행동은어떤가
Package Health / SCA	security, maintenance, popularity, quality	새선택/유지/이탈의시간흐름은어떤가
Migration Research	declining package 의 alternative/migration pattern 탐색	추천이전에현재 behavior 와 evidence confidence 를어떻게설명할까

Product Definition

Vision

공개 OSS dependency history 에서실제선택·유지·이탈·이동행동을재현가능한근거로계산하고, Tech Lead 가신규핵심 dependency 를그대로기술검토할지 alternatives 를먼저비교할지판단하도록돕는다.

Product Boundary

제공	제공하지않음
Adoption / Retention / Exit	Best package 추천
Cohort-relative behavior	미래 survival prediction
Transition Candidate / repeated flow	자동 migration 코드
Evidence Confidence	Vulnerability / License scanner
Repository-supported examples	전체 JavaScript ecosystem 추정
Proceed / Compare / Investigate	실제 runtime usage 추정

제품의핵심문장은다음과같다.

OSS Shift analyzes where an OSS ecosystem is actually staying, leaving, and moving - before your team commits to the dependency.

데이터와분석계약

무엇을관찰하고무엇을계산하는지먼저고정한다. Data Contract 가없으면 State 와 Migration 해석이흔들린다.

Observation Contract

NPM_REGULAR_DEPENDENCY_V1

관찰모집단은 deps.dev 의 selected latest snapshot 에서관찰가능한 published npm package versions 다.

Release eligibility 는다음과같다.

```
System = 'NPM'
VersionInfo.IsRelease = TRUE
UpstreamPublishedAt IS NOT NULL
NPMRequirements state observable
```

MVP dependency scope 는 NPMRequirementsLatest.Dependencies 하나다. DevDependencies, OptionalDependencies, PeerDependencies, BundledDependencies 는제외한다.

따라서제품에서사용하는가장정확한표현은 **regular dependency declaration**이다. 실제 runtime code path 에서사용된다는의미가아니다.

deps.dev Schema Verification

2026-08-20 기준공식 deps.dev BigQuery 문서와실제 BigQuery public dataset query 를교차확인했다. NPMRequirements 와 NPMRequirementsLatest 가존재하며, npm requirement 에는 Dependencies, DevDependencies, OptionalDependencies, PeerDependencies 가구분되고각 requirement 에 package name 과 requirement string 이제공된다. NPMRequirementsLatest 는최신 snapshot view 다.

Actual Data Feasibility

npm Release Population

이번프로젝트에서실제실행한 BigQuery profiling 결과다.

Metric	Observed value
Release versions	46,603,902
Distinct packages	4,024,046
DependenciesProcessed = true	46,389,825
DependencyError = true	1,957,570
DependencyError ratio	약 4.20%

DependencyError=true 인 version 도이번 profiling 에서 NPM requirements row 와 regular dependency declarations 가대부분존재했다. 따라서 DependenciesProcessed 와 DependencyError 는 **QA metadata** 로보존하되 raw declaration observation 을자동제외하는기준으로사용하지않는다.

Missing 과 Empty 를구분한다

이번 profiling 조건에서 release versions 는모두 NPMRequirementsLatest 와연결되었다. 그러나 pipeline contract 는미래 snapshot anomaly 에대비해다음의미를유지한다.

상태	해석
Requirements row 존재 + Dependencies = []	OBSERVED EMPTY: regular dependency declaration 0 개
Requirements row 자체가 없음	UNKNOWN: dependency state 미관측, Exit 계산금지

UNKNOWN 을 EMPTY 로바꾸면데이터미관측이가짜 Exit 로변할수있다.

Release History Viability

package 당 release 수는 p50=3, p75=8, p95=40, p99=156, max=37,328 이었다. 두개이상의 release 를가진 package 는 2,578,837 개, 약 64.1% 였다.

이결과는두가지를동시에보여준다.

- 연속 release 기반 behavior 분석의후보 population 이충분하다.
- release frequency 가극단적으로높은 package 가존재하므로 release count 를그대로 ecosystem vote 로사용하면 bias 가생길수있다.

따라서 State 계산은뒤에서설명하는 **package-period collapse**를사용한다.

HTTP_CLIENT_V1 Support

Package	Declarations	Distinct dependents	2020+	2022+
axios	2,579,970	166,297	2,327,670	1,862,755
request	554,489	51,590	185,461	82,176
node-fetch	640,809	45,236	557,975	417,488
superagent	128,439	9,072	65,186	38,797
got	127,335	8,814	106,877	71,810
undici	110,358	7,522	110,339	109,622

이수치는현재 adoption 을의미하지않는다. 기존 adopter 가새 version 을 publish 하면같은 dependency declaration 이다시관찰될수있다. 바로이이유때문에 Adoption 과 Retention 을분리한다.

Data Feasibility: PASS

package-version history, regular dependency declaration, release-requirements join, consecutive release population, HTTP client cohort support 가모두확인되었다.

Analysis Unit and Mainline

원자단위

dependent package P 의 mainline releases 를 $r_1, r_2, \dots, r_n$, release r_i 의 regular dependency set 을 $D(P, r_i)$ 라고한다. 기본 observation 은연속 release transition 이다.

$$T_i = (r_{i-1} \rightarrow r_i)$$

예를들어 $D(P, r_1) = \{A, C\}$, $D(P, r_2) = \{B, C\}$ 이면 A 는 REMOVED, B 는 ADDED, C 는 RETAINED 다. 이 단계에서는 A 에서 B 로 migration 했다고부르지않는다.

Mainline Release Approximation

publish time 만사용하면과거 branch backport 가섞일수있다. MVP 에서는 time order 를따라가며 current ordinal 이이전 running maximum 보다클때만 mainline 으로인정한다.

$$isMainline(r_i) = \mathbf{1} \left[Ordinal(r_i) > \max_{j < i} Ordinal(r_j) \right]$$

이 rule 은 parallel major maintenance 를완전히모델링하지않는다. 따라서 **MVP Mainline Approximation**으로명시하고 representative package history 에서별도 validation 한다.

Behavior Metrics

Change Event

두연속 mainline release 사이에서 dependency A 의상태를비교한다.

Event	정의
RETAINED	$A \in D_{prev}$ and $A \in D_{curr}$
REMOVED	$A \in D_{prev}$ and $A \notin D_{curr}$
ADDED	$A \notin D_{prev}$ and $A \in D_{curr}$
REQUIREMENT_CHANGED	앞쪽에존재하지만 requirement string 변경

Requirement change 는 migration 이나 adoption 으로취급하지않는다.

Entry Adoption 과 Switch-in Adoption

Entry Adoption은 첫 observable mainline release 에 A 가 이미 존재하는 경우다. 실제 새 프로젝트가 그 시점에 A 를 처음 선택했다는 뜻은 아니다.

Switch-in Adoption은 이전 observable release 에는 A 가 없고 다음 release 에는 A 가 생긴 경우다. 이것이기 존 published package 의 신규 declaration behavior 를 더 직접적으로 나타낸다.

Retention 과 Exit

기간 t 시작시 A 를 가지고 있었고 그 기간에 새 mainline release 가 실제로 관찰된 distinct dependent package 만 denominator 에 넣는다.

$$RetentionRate_{A,t} = \frac{Retained_{A,t}}{EligibleExisting_{A,t}}$$

$$ExitRate_{A,t} = \frac{Exit_{A,t}}{EligibleExisting_{A,t}}$$

핵심 원칙은 새 release 가 없음 $\neq$ Exit 이다.

Package-Period Collapse

release frequency bias 를 줄이기 위해 State 계산에서는 package P 의 period start 와 period end dependency state 만 사용한다. 한 달에 release 가 30 개 있어도 package-dependency-period 당 최대한 observation 으로 collapse 한다. 중간 release-level event 는 Evidence 와 Transition 분석을 위해 그대로 보존한다.

Cohort-relative Metrics

기간 t 에 mainline release 를 낸 전체 distinct npm package 수를 P_t 라고 할 때, A switch-in 의 absolute intensity 는 다음과 같이 둔다.

$$SwitchInIntensity_{A,t} = \frac{SwitchIn_{A,t}}{P_t}$$

HTTP client cohort C 의 switch-in event 중 A 가 차지하는 상대 비중은:

$$CohortSwitchInShare_{A,t} = \frac{SwitchIn_{A,t}}{\sum_{c \in C} SwitchIn_{c,t}}$$

또 기간 끝에 A 를 가진 active publishing package 수를 $U_{A,t}$, cohort member 하나 이상을 가진 package 수를 $U_{C,t}$ 라 하면:

$$CohortPresenceShare_{A,t} = \frac{U_{A,t}}{U_{C,t}}$$

이 값은 market share 가 아니라 **curated cohort 안에서의 observed presence**다.

Cohort Contraction 을위한 Footprint

cohort 자체가 observable npm publishing population 에서줄어드는지보기위해:

$$CohortFootprint_t = \frac{C_t}{P_t}$$

를사용한다. C_t 는해당 period 에 HTTP_CLIENT_V1 하나이상을 declaration 한 active publishing package 수다. 이값의감소를전체 Node.js HTTP 시장감소로일반화하지않는다.

Transition Contract

Transition Candidate

같은 release transition 에서 A 가 REMOVED 되고 B 가 ADDED 되면:

$$TransitionCandidate(P, A, B, t)$$

를만든다. 의미는 **same-release co-change correlation**까지이며 migration intent 가아니다.

Pair Complexity

한 transition 에서 removed dependency 가 m 개, added dependency 가 n 개라면 potential pair 는:

$$PairComplexity = m \times n$$

이다. Pair Complexity 가커질수록 unrelated co-change false positive 와 candidate row expansion/shuffle 이함께증가한다. 따라서이후 quality gate 는제품정확성과 distributed cost 를동시에개선하는지점이된다.

No Observed Replacement

A 가제거되었지만같은 transition 에서다른 HTTP_CLIENT_V1 package 가추가되지않으면 **NO_OBSERVED_COHORT** 기록한다.

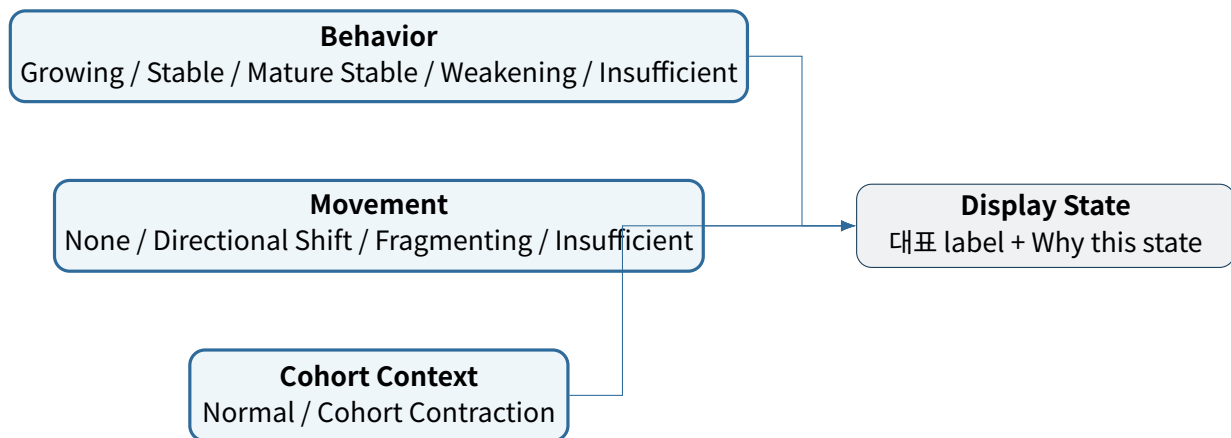
No Observed Replacement 는 Native Migration 이아니다. node-fetch removal 이후 native fetch 를사용했는지여부는 npm dependency declaration 만으로확인할수없다. 강한해석은 repository evidence 가필요하다.

State & Evidence

State 를 점수로 만들지 않는다. Behavior, Movement, Cohort Context 를 분리하고, rule/metric/period/support 를 모두 재현 가능하게 남긴다.

State Model

기존 후보 State 는 서로 다른 현상을 한 축에 섞을 위험이 있다. 따라서 내부 계산을 세 축으로 나눈다.



State 와 Evidence Confidence 도 별도로 유지한다. 예를 들어 **Directional Shift / Confidence: Medium** 처럼 표현한다.

Time, Baseline and Support

Period Granularity

raw aggregate 는 calendar month 를 기본 primitive 로 저장한다. State evaluation 은 EDA 결과에 따라 cohort 전체에 공통으로 MONTHLY 또는 QUARTERLY 를 선택한다. package 마다 서로 다른 granularity 를 쓰지 않는다.

선택 기준은 eligible denominator, rate confidence interval, low-support period 비율, bootstrap State stability 다.

Baseline 과 Trend

metric x_t 에 대해 baseline 과 recent level 을 robust median 으로 정의한다.

$$Baseline(x) = Median(x_t, t \in B), \quad Recent(x) = Median(x_t, t \in R)$$

$$\Delta_x = Recent(x) - Baseline(x)$$

장기방향성은 Theil-Sen 계열 robust slope β_x 를 사용하고, baseline noise scale 은 MAD 계열을사용한다. 단한 period 의 spike 가 State 를뒤집지않도록하기위한설계다.

UP / DOWN / NEUTRAL Primitive

State rule 은 raw 숫자를직접비교하지않고다음 primitive 를사용한다.

$$UP(x) = [\Delta_x > \delta_x] \wedge [\beta_x > \beta_x^{min}] \wedge SupportOK$$

$$DOWN(x) = [\Delta_x < -\delta_x] \wedge [\beta_x < -\beta_x^{min}] \wedge SupportOK$$

그외는 NEUTRAL 이다. δ 와 slope threshold 는임의로고정하지않고 calibration 한다.

Minimum Support

$2/4 = 50\%$ 와 $5,000/10,000 = 50\%$ 를같은 evidence 로취급하지않는다. high-support historical window 를 down-sampling 하며 confidence interval width, trend sign stability, bootstrap State agreement 가안정되기시작하는 sample size 를 minimum support 후보로정한다.

Movement Mathematics

Replacement Coverage

A Exit 중 quality-gated same-cohort destination 이관찰된 Exit 의비율을:

$$ReplacementCoverage_A = \frac{ReplacementExit_A}{AllObservableExit_A}$$

로둔다. 이값이낮다면 destination distribution 만으로강한 Movement 를주장하지않는다.

Destination Distribution

quality-gated A 에서 B 로의 flow 를보인 distinct dependent package 수를 n_{AB} 라하면:

$$q_B = \frac{n_{AB}}{\sum_j n_{Aj}}$$

$$TopDestinationShare_A = \max_B(q_B)$$

TopDestinationShare 는전체 Exit 중비율이나니라 eligible destination distribution 안에서의 concentra-tion 이다. 따라서 Replacement Coverage 와항상함께보여준다.

Entropy 와 HHI

$$H(A) = - \sum_B q_B \log q_B$$

$$H_{norm}(A) = \frac{H(A)}{\log K}, \quad HHI(A) = \sum_B q_B^2$$

H_{norm} 이 낮고 HHI 가 높으면 한 destination 으로 집중되는 경향, H_{norm} 이 높고 HHI 가 낮으면 여러 destination 으로 분산되는 경향을 의미한다.

No Observed Replacement 는 destination entropy 에 넣지 않고 별도 비율로 유지한다.

$$NoReplacementShare_A = \frac{NoReplacementExit_A}{AllObservableExit_A}$$

State Rules

Stable

BEHAVIOR_SUPPORT_OK
AND NEUTRAL(CohortPresenceShare)
AND NEUTRAL(CohortSwitchInShare)
AND NEUTRAL(ExitRate)
AND RetentionRate in calibrated stable range

의미는 뚜렷한 strengthening 또는 weakening signal 이 관찰되지 않는다는 것이며, 좋은 package 라는 보증이 아니다.

Growing

UP(CohortPresenceShare)
AND (UP(CohortSwitchInShare) OR DOWN(ExitRate))

observable cohort 안에서 A 의 선택/유지 behavior 가 상대적으로 강화되는 경우다.

Mature Stable

Stable 의 subset 으로, 높은 Retention, 낮은 Exit, 낮지만 붕괴하지 않는 신규 선택, 그리고 npm package release cadence 가 calibrated mature band 에 있는 경우다. GitHub commit frequency 는 Core State 에 넣지 않는다. GitHub coverage 차이가 State 를 왜곡하면 안 되기 때문이다.

Weakening

한 signal 하나로 판정하지 않는다.

DOWN(CohortPresenceShare) AND [DOWN(CohortSwitchInShare) OR UP(ExitRate)]
OR
 DOWN(CohortSwitchInShare) AND UP(ExitRate)

즉적어도 두개의 independent weakening signals 을 요구한다.

Directional Shift

Weakening behavior 에 더해 Exit 증가, sufficient Movement support, sufficient Replacement Coverage, 높은 TopDestinationShare, 낮은 entropy, 높은 HHI, independent evidence gate 를 모두 통과해야 한다.

제품문장은 “A Exit 와 같은 release 에서 특정 B 가 추가되는 repeated functional flow 가 한 destination 으로 집중되어 관찰됨” 까지다. intentional migration 을 자동 주장하지 않는다.

Fragmenting

Weakening 과 Exit 증가가 존재하고, sufficient Movement support 와 Replacement Coverage 를 확보했지만 destination 이 여러 곳으로 퍼져 높은 entropy, 낮은 HHI, 낮은 TopDestinationShare 를 보이면 Fragmenting 으로 본다. Directional 과 Fragmenting 사이 gray zone 이면 억지로 label 을 고르지 않고 Weakening 으로 남긴다.

Cohort Contraction

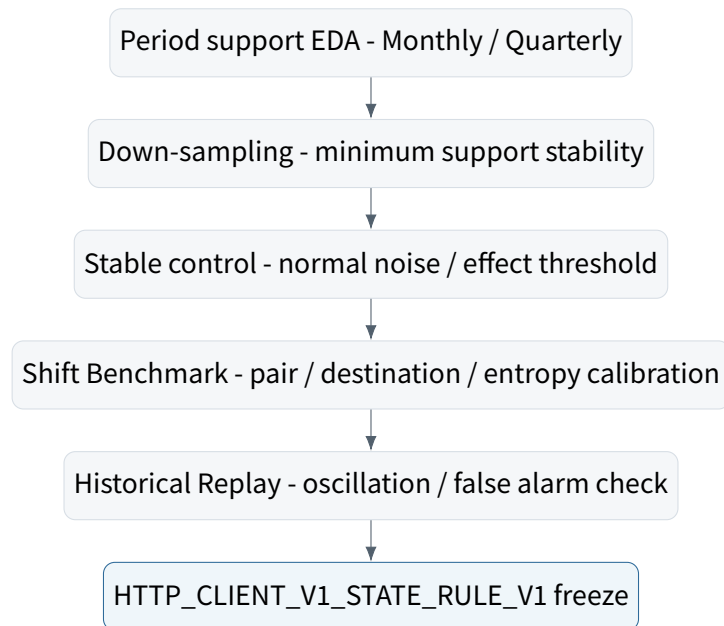
CohortFootprint 가 하락하고 cohort-wide 신규선택 intensity 감소 또는 cohort-wide Exit 증가가 함께 나타날 때 context 후보가 된다. focal A 의 relative CohortPresenceShare 가 cohort 전체보다 유의하게 더 나쁘지 않을 때 Display State 를 Cohort Contraction 으로 올린다.

Insufficient Evidence

recent/baseline coverage, eligible Exit denominator, Switch-in support, cohort denominator, core DQ 중 하나가 calibrated minimum 에 미달하면 Behavior 를 보류한다. Movement support 만 부족하면 Behavior 까지 제거하지 않고 **Weakening / Movement Insufficient** 처럼 표현한다.

Threshold Calibration

threshold 는 결과에 맞추어 임의로 조정하지 않는다.



Week 4 이후에는 결과가 마음에 들지 않는다는 이유로 threshold 를 계속 바꾸지 않는다.

Evidence Confidence and Ladder

Evidence Confidence

State 와 분리해 Support Sufficiency, Time Coverage, Independent Dependents, Project Diversity, Pair Quality, Repository Evidence 를 관리한다. project mapping 은 authoritative 하지 않을 수 있으므로 보조 diversity evidence 로만 사용한다.

Evidence Ladder

Level	의미
Observed Change	한 release transition 의 add/remove
Transition Candidate	A removed + B added co-change
Repeated Flow	여러 independent dependent 에서 반복
Functional Transition	curated functional cohort 안에서 반복
Repository-supported	실제 manifest/PR/commit instance 가 구조와 일치
Explicit Migration Example	해당 repository evidence 에 교체의도/이유가 명시

마지막 단계도 ecosystem 전체 pair 가 confirmed migration 이라는 뜻이 아니다.

AI Evidence Interpreter

AI 는 Adoption, Retention, Exit, Transition count, State, package recommendation 을 계산하지 않는다.
selected GitHub evidence 의 비정형의도와 이유만 구조화한다.

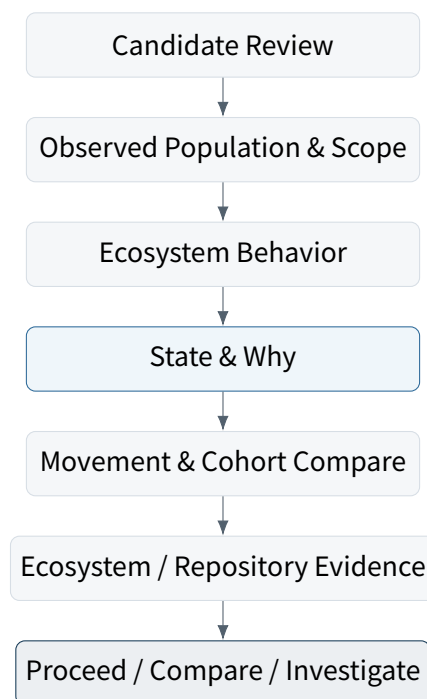
```
migration_intent: true | false | unclear  
reason_type: DEPRECATION | MAINTENANCE | PERFORMANCE | API | COMPATIBILITY | SECURITY | OTHER  
| NOT_STATED  
reason_summary: string | null  
evidence_span: string | null  
unsupported_inference: boolean
```

evidence_span=null 이면 reason 을 숨기고, unsupported_inference=true 이면 AI interpretation 을 제품에서 억제한다.

Product Experience

State 를보여주는데서끝나지않고, Tech Lead 가왜이결과가나왔고무엇을더검토해야하는지이해하게 만든다.

End-to-End Flow



Behavior and Why This State

핵심 behavior surface 는 Switch-in Adoption, Retention, Exit, Cohort Presence Share 다. 각 metric 에 는 Recent, Baseline, Direction, Support 를함께보여준다.

State label 만보여주지않고 triggered rule 을노출한다.

예시 - Why Weakening?

Cohort Presence Share: DOWN

Exit Rate: UP

Cohort Switch-in Share: NEUTRAL

Rule: 2 independent weakening signals detected

Rule Version: HTTP_CLIENT_V1_STATE_RULE_V1

제안서의수치는 UI 구조설명용 mock 일수있지만실제제품 State 는 frozen rule output 만사용한다.

Movement and Cohort Compare

Movement 화면은 destination 비율뿐아니라 Replacement Coverage, Top Destination Share, Normalized Entropy, HHI, Independent Dependents 를같이보여준다.

A Exit Movement - illustrative

B 51% C 18% D 8% No observed cohort replacement 23%

Same-release remove/add flow is correlation evidence. It does not by itself establish intentional migration.

Cohort Compare 는 0-100 health score 가아니라 behavior 를나란히비교한다.

Package	New Selection	Retention	Exit	Cohort Position	State
A	Down	High	Up	Down	Weakening
B	Up	High	Stable	Up	Growing
C	Stable	High	Low	Stable	Mature Stable

Review Action Contract

Display State	Default Action	의미
Growing / Stable / Mature Stable	Proceed to Review	강한 shift signal 없이일반기술검토지속
Weakening / Directional / Fragmenting Cohort Contraction	Compare Alternatives Investigate	architecture commitment 전에 cohort alternatives 비교 focal package 보다 category context 추가확인
Insufficient Evidence	Investigate	판단근거부족

Confidence 가 Low 이면 State 기반 action 을 **Investigate**로 downgrade 할수있다. Action 은 recommendation 이아니라 review workflow 다.

Limitations at the Point of Decision

Limitation 은 별도 약관 페이지에 숨기지 않고 결과가 가까이 에둔다.

항목	제품이 말할 수 없는 것
Observed Population	모든 JavaScript applications / private repos / internal software
Dependency Declaration	실제 runtime code path usage
Transition Candidate	intentional migration 또는 causality
No Replacement	native API migration
HTTP_CLIENT_V1	완전한 HTTP market taxonomy
State	future package survival / best library choice

Distributed System

데이터크기를과장하지않는다. 제품계산에서실제로발생하는 Window, Join, Shuffle, Expansion, Aggregation 을 multi-worker 에서재현하고최적화효과를측정한다.

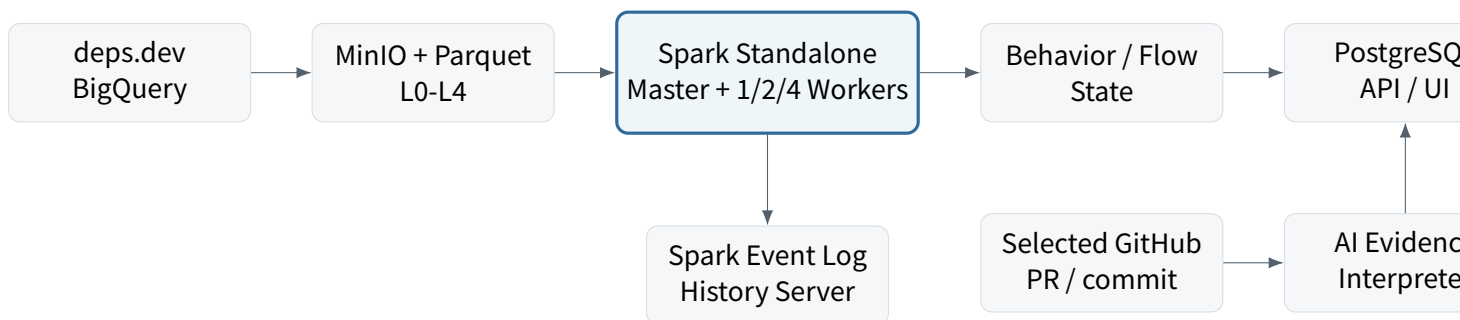
Distributed Design Principle

OSS Shift는 “Spark 없이는불가능하다” 고주장하지않는다. 대신약 4,660 만 release 의 dependency history 에서 package 별 release sequencing, large join, dependency diff, removed-by-added expansion, dependency/pair aggregation 이실제제품계산에필요하다는점을근거로분산시스템을설계한다.

Distributed Proof 의질문

같은 logical workload 를 1/2/4 Worker 에서실행했을때어디에서 shuffle, spill, skew, serialization overhead 가발생하며 Partition, Broadcast, AQE, Pair Quality Gate 가처리비용을어떻게바꾸는가?

System Architecture



BigQuery 는 source extraction/profiling, Spark 는제품계산, PostgreSQL 은 serving 을담당한다. GitHub 는 selected evidence layer 이며 ecosystem 전수 source 가아니다.

Data Lake and Partition Contract

Layer	Content	Physical Partition
L0 RAW	deps.dev snapshot export	snapshot_date
L1 RELEASE	release timeline input	published year/month
L1 REQUIRE-MENT	regular declarations	published year/month
L2 CHANGE	add/remove/retain/change	event year/month
L3 TRANSITION	quality-gated candidate flow	event year/month
L4 AGGREGATE	behavior/flow/state	period

package name 은 physical directory partition 으로 쓰지 않는다. 402 만 package 의 high cardinality 는 small-file explosion 위험이 크다. Physical partition 은 scan 범위를 줄이는 storage layout 이고 Spark repartition 은 runtime 계산을 위한 row redistribution 이다.

Spark Job DAG

Job	Input - Output	Main operation	Distributed character
D1 Release Timeline	Release - Timeline	Window / running max / lag	Wide
D2 Requirement State	Timeline + Requirements	large-large join	Wide
D3 Dependency Diff	Prev/Curr - Change	FULL OUTER JOIN	Heavy Wide
D4 Pair Generation	Removed x Added	self join / m x n expansion	Wide + amplification
D5A Period Collapse	Changes - package-period	Window / aggregate	Wide
D5B Behavior Aggregate	package-period - A/R/E	GroupBy	Wide + skew candidate
D6 Flow Aggregate	Candidate - pair flow	distinct / GroupBy	Wide + skew candidate
D7 State Detect	aggregate - State	temporal Window / rule	reduced data
D8 Evidence	selected pair - GitHub	limited external process	outside core Spark
D9 Publish	validated result - DB	batch publish	serving boundary

핵심 Wide Stage

D1 은 package-level release Window, D3 는 transition/dependency FULL OUTER JOIN, D4 는 removed-by-added expansion, D5/D6 는 dependency/pair GroupBy 에서 주요 shuffle 이 발생한다. D5A 는 Phase 3 의 package-period collapse 를 실제 DAG 에 반영한 단계다.

Optimization Strategy

Pair Quality Gate A/B

D4 에서 All Removed x Added 와 Quality-gated candidate generation 을 비교한다. candidate rows, shuffle write, stage time, spill, manual transition precision 을 함께 본다.

이 실험이 중요한 이유는 **제품 정확성 gate** 와 **distributed cost optimization** 이 같은 지점에서 만난다는 데 있다.

Broadcast / AQE / Skew

작은 cohort_registry, package_relation_registry 는 Broadcast Join 후보다. AQE 는 OFF/ON 을 비교하고 runtime partition coalescing 과 skew optimization 효과를 확인한다.

Skew 대응 순서는 고정한다.

Baseline - AQE - actual hot key 확인 - 필요할 때만 Salting 또는 2-stage aggregation

실제 skew 가 없으면 manual salting 을 하지 않는 것이 올바른 결과다.

Scale-Out Benchmark and Correctness

동일 input, code, configuration 에서 Worker 1/2/4 를 비교한다. Worker 당 cores/memory 는 동일하게 두고 Worker 증가에 따라 총 compute resource 가 늘어나는 scale-out experiment 를 수행한다.

측정 metric:

- wall time, records/sec, executor-hour
- stage wall time, task count, p50/p95 task duration
- shuffle read/write, fetch wait, memory/disk spill
- executor CPU/GC, partition distribution

선형 speedup 은 성공 조건이 아니다. 2 Worker 이후 overhead 가 커져 4 Worker 가 느려진다면 crossover 를 시스템 결과로 설명한다.

성능보다 먼저 지켜야 할 correctness invariant 는 다음과 같다.

$$Output_{1W} = Output_{2W} = Output_{4W}$$

$$Output_{AQE\ Off} = Output_{AQE\ On}, \quad Output_{Broadcast\ Off} = Output_{Broadcast\ On}$$

canonical sorting/serialization 뒤 checksum 으로 dependency_change, package_period_behavior, pair_period_flow, package_shift_state 를 비교한다.

Observability

Spark UI / Event Log / History Server 는장식이아니라분산증거저장소다. 각 benchmark run 에 snapshot id, code version, worker count, Spark config, state rule version 을남기고 Job/Stage DAG, executor task distribution, shuffle, spill, SQL physical plan 을캡처한다.

Validation & Execution

결과를만들기전에성공과실패기준을정의한다. Meaningful Movement 가사라지면억지로살리지않고 Lifecycle 중심으로 Pivot 한다.

Validation Framework

검증을하나의 accuracy 숫자로합치지않는다.

Layer	핵심질문
Data Validity	dependency history 자체를믿고계산할수있는가
Behavior Correctness	Adoption / Retention / Exit 가 source 와일치하는가
Movement Specificity	co-change noise 가 top result 를지배하지않는가
State Reliability	State 가재현가능하고 noise 에과민하지않는가
Evidence Validation	AI 가 source 밖 inference 를만들지않는가
Product Validation	Tech Lead 가 State/Transition 의미를오해하지않는가
Distributed Proof	same output 을 multi-worker 에서재현하고비용을설명하는가

Quality Gates

Data and Change Gates

snapshot 마다 Release/Requirements join coverage, Missing vs Empty, publish timestamp, ordinal anomalies, npm alias, snapshot consistency 를확인한다. 20-30 representative transition 을 source declaration 과 manual 비교해 add/remove/retain/requirement change 가 deterministic 하게일치해야 한다.

Mainline rule 은 normal semver, major transition, later backport, parallel major maintenance, unusual release cadence 를포함한 sample history 에서검증한다.

Shift Benchmark Set

Movement 는 50-80 transition instance 로 precision-first validation 한다.

- Known migration-like positive
- Co-change negative
- No-Replacement / Native candidate
- Stable / Mature control

pair complexity, same cohort, independent dependents, Replacement Coverage, Top Destination Share, Entropy, HHI, owner/project concentration, repository evidence 를 비교한다.

Movement에서는 **Recall** 보다 **Precision** 을 우선한다. 애매한 flow 를 숨기는 것이 잘못된 migration-like claim 을 강하게 보여주는 것보다 낫다.

State Reliability

Historical Replay 에서 의미없는 Stable/Weakening oscillation 이 반복되지 않는지, bootstrap resampling 에서 State agreement 가 안정적인지, Stable/Mature controls 에서 false alarm 이 과도하지 않은지 확인한다.

모든 State result 는 display state, triggered rules, recent/baseline metrics, support, rule version 을 가져야 한다. 사람이 같은 metric 으로 결과를 재현할 수 없으면 fail 이다.

Repository / AI Evidence Gate

selected pair 의 실제 PR/commit 을 사람이 읽고 AI output 을 검토한다. 가장 치명적인 실패는 reason taxonomy 오분류보다 **source** 에 없는 이유를 생성하는 것이다. Evidence span 이 없거나 unsupported inference 가 발생하면 제품에서 AI interpretation 을 숨긴다.

Go / Conditional / Pivot

Decision	조건	조치
GO	history, A/R/E, meaningful repeated flow, State explainability, evidence 확보	현재 방향 유지
CONDITIONAL	transition noise 가 높지만 gate 로 통제가 가능	cohort/complexity/diversity 강화
PIVOT	quality gate 이 후 meaningful movement 가 거의 사라짐	Movement 를 Core 에서 내리고 Ecosystem Lifecycle Intelligence 로 전환
NO-GO	declaration history 또는 A/R/E 자체 가 재현 불가	데이터/분석 방향 재설계

Lifecycle Pivot 의 Core 는 Adoption, Retention, Exit, Cohort Position, Mature Stable, Weakening, Cohort Contraction 이다. Transition 이 실패해도 behavior layer 의 제품 가치는 별도 검증한다.

Six-Week WBS

Week	Goal	Main Work	Completion Evidence / Gate
1	Data Contract & Distributed Smoke	snapshot 고정, profiling, raw parquet, Spark Master + 2 Worker smoke	Snapshot Manifest, Data Contract, History log. history 불가시 기간축소또는 No-Go
2	Behavior Engine	Mainline, Requirement State, Diff, Package-Period Collapse	20-30 manual QA, A/R/E sample. sequencing/diff 불안정시수정후재 검증
3	Movement & State Calibration	Pair generation, 50-80 benchmark, threshold calibration, selected GitHub sample	Transition specificity + State Rule candidate. 가장큰 Go/Pivot 지점
4	Product E2E / Scope Freeze	aggregate, State/Action, PostgreSQL, API, Candidate/Behavior/Movement/Evidence UI	Candidate-to-Decision E2E. 이후 신규분석기능추가금지
5	Distributed Proof	1/2/4 Worker, Pair Gate, Broadcast, AQE, actual skew observation	benchmark + History evidence. manual salting 은필요할때만
6	Final Validation	correctness replay, AI QA, developer review, final benchmark, demo	Final Acceptance Package. 미검증 기능은숨김

Team and Parallel Development

Role	Responsibility
PM / Product	Problem, State Contract, Validation, Go/Pivot, Demo
AI + UI/UX	Evidence Interpreter, Evidence/Decision UX
Data Engineer	deps.dev extraction, DQ, release sequencing
Distributed Data Engineer	Diff, Pair, Aggregate, optimization, benchmark
Backend / Infra	Spark Standalone, MinIO, PostgreSQL, History Server, API
Frontend	Candidate, Behavior, Movement, Evidence, Decision surfaces

Frontend 는실제 State result 를기다리지않고 Week 1-2 부터 mock contract 로병렬개발한다.

Risk and Scope Cut

Risk	Failure Signal	Response
Mainline distortion	backport 가 prev release 를 지배	sample validation / scope 제한
Transition noise	unrelated co-change 가 top pair 지배	complexity/cohort/diversity gate
Movement 거의없음	quality gate 뒤 pair 소멸	Lifecycle Pivot
Sparse State	monthly denominator 부족	Quarterly
Owner concentra- tion	flow 대부분의 owner/project	Movement confidence 하향/보류
AI hallucination	source 에 없는 reason	evidence-span gate
Spark skew	p95 task, spill, straggler 상승	AQE - 필요시 salting
4 Worker slower	overhead 가 compute gain 보다 큼	crossover 로 설명
Small files	excessive objects/tasks	partition 축소 / compaction
Scope expansion	새 source/tool 요구	Week 4 freeze

일정지연시 AI Review Brief, GitHub evidence 개수, 고급 Cohort Compare/Fragmentation UX, front-end polishing, AI reason taxonomy, multi-host 를 먼저 줄인다. dependency history, Adoption/Retention/Exit, Transition Candidate, quality gate, Spark 2+ Worker, Shuffle evidence, correctness 는 자르지 않는다.

Final Acceptance Criteria

Product

Candidate - Behavior - State - Movement - Evidence - Review Action E2E 가 동작하고 Proceed / Compare / Investigate 중 하나가 deterministic 하게 생성되어야 한다.

Analysis

Entry/Switch-in 을 구분하고, Retention/Exit denominator 와 Mainline rule 을 명시하며, Requirement Change 를 별도 처리하고, No Replacement limitation 과 triggered State rule 을 추적할 수 있어야 한다.

Movement and Evidence

50-80 benchmark set 에서 unrelated co-change 가 top result 를 지배하면 Movement fail 이다. demo result 는 source package/release 까지 trace 가능하고 repository evidence 를 사용한 경우 PR/commit 과 evidence span 까지 연결되어야 한다.

Distributed

Spark Standalone 2+ Worker 실제 task 참여, 1/2/4 Worker benchmark, actual shuffle metric, 최소 2 개의 optimization A/B, logical output checksum equality 를 반드시 남긴다.

Honesty

제품과 발표에 다음한계를 고정한다.

- Published npm package ecosystem $\neq$ all JavaScript applications
- Dependency declaration $\neq$ runtime usage
- Transition Candidate $\neq$ migration intent
- HTTP_CLIENT_V1 $\neq$ complete HTTP market taxonomy
- State $\neq$ future survival prediction

Final Demo Narrative

최종 demo 는 illustrative State 를 실제 결과처럼 연기하지 않는다. 실제 pipeline result 가 확보된 package 를 사용한다.

1. Tech Lead 가 신규 core dependency 후보를 입력한다.
2. Observed Population 과 limitation 을 먼저 확인한다.
3. Recent vs Baseline Behavior 를 본다.
4. State 와 triggered rules 를 확인한다.
5. Movement 가 있으면 Replacement Coverage, concentration, No Replacement 를 확인한다.
6. release-level source evidence 를 연다.
7. selected GitHub evidence 가 있으면 AI 가 evidence span 안에서 intent/reason 을 구조화한다.
8. Review Action 을 확인한다.
9. Engineering Proof 로 전환해 1/2/4 Worker 결과와 핵심 Shuffle Stage 를 보여준다.
10. Pair Quality Gate 또는 Broadcast/AQE 전후를 비교하고 correctness checksum 이 동일함을 확인한다.

Appendix & Sources

핵심용어와외부사실의근거를분리해문서의재현성과검증가능성을높인다.

Metric Dictionary

Metric / Term	Definition
Entry Adoption	첫 observable mainline release 에 A 가이미존재. 새프로젝트탄생으로해석금지
Switch-in Adoption	이전 observable release 에는없던 A declaration 이다음 release 에추가
Retention	A 를가지고 period 를시작한 eligible dependent 가 period end 에도 A 유지
Exit	A 를가지고 period 를시작한 eligible dependent 의다음 observable state 에서 A 제거
Cohort Presence Share	HTTP_CLIENT_V1 하나이상을가진 active package 중 A presence 비율
Cohort Footprint	active npm publishing package 중 HTTP_CLIENT_V1 하나이상을가진비율
Transition Candidate	같은 release transition 에서 A removed + B added
Pair Complexity	한 transition 의 removed count x added count
Replacement Coverage	observable A Exit 중 eligible same-cohort destination 을가진 비율
Top Destination Share	eligible destination distribution 안에서가장큰 destination 비율
Normalized Entropy HHI	destination distribution 의분산정도, 0-1 normalized destination concentration 보조지표
No Replacement Share	A Exit 중 same-cohort destination 이관찰되지않은비율
Evidence Confidence	support/time/diversity/pair quality/repository evidence 의 별도 confidence layer

MVP In / Out

IN	OUT
npm / HTTP_CLIENT_V1	PyPI / Maven
Adoption / Retention / Exit	Kafka / GraphFrames
Transition Candidate / State	Kubernetes / Airflow
Evidence Confidence	full GitHub crawling
selected GitHub evidence	vulnerability/license scanner
AI evidence interpretation	Health Score / best recommender
Batch Spark / 1-2-4 Worker	future survival prediction
	automatic migration code / row-level LLM

Source and Verification Notes

외부시스템의 capability 와 schema 는 공식문서를 우선하며, 실제 feasibility 숫자는 프로젝트가 직접 실행한 BigQuery profiling 결과와 구분한다.

- **deps.dev BigQuery dataset:** schema, PackageVersions, NPMRequirements/NPMRequirements-Latest, PackageVersionToProject limitation.
<https://docs.deps.dev/bigquery/v1/>
- **Apache Spark Standalone Mode:** Master/Worker cluster 구성.
<https://spark.apache.org/docs/latest/spark-standalone.html>
- **Apache Spark SQL Performance Tuning:** broadcast, AQE, shuffle/skew optimization.
<https://spark.apache.org/docs/latest/sql-performance-tuning>
- **Apache Spark Monitoring:** Event Log, Web UI, History Server.
<https://spark.apache.org/docs/latest/monitoring.html>
- **GitHub Dependency Review:** repository-level dependency changes 와 security impact review.
<https://docs.github.com/en/code-security/concepts/supply-chain-security/dependency-review>

Actual Project Profiling Note

2026-08-20 Google BigQuery 에서 bigquery-public-data.deps_dev_v1 을 대상으로 실행한 profiling 결과를 본문 Data Feasibility 에 사용했다. 이 값은 해당 실행시점의 Latest view 관찰값이며, 향후 snapshot 에서 다시 측정한다.

최종한문장

OSS Shift 는 “죽어가는 OSS 를 맞이하는 서비스” 가 아니라, 공개 dependency history 에서 실제 선택·유지·이탈·이동 행동을 근거와 함께 보여주고, 그 계산을 multi-worker 분산 시스템으로 구현·검증하여 Tech Lead 의 신규 dependency review 를 돕는 프로젝트다.